

Top 20 JavaScript Questions for Junior Developers (2026)

Curated interview questions covering fundamentals every junior JavaScript developer should know in 2026 — including modern ES2024+ features employers expect you to recognize.

Table of Contents

1. [What are the data types in JavaScript?](#)
2. [What is the difference between `let`, `const`, and `var`?](#)
3. [What is the difference between `==` and `===`?](#)
4. [What is hoisting?](#)
5. [What is a closure?](#)
6. [How does the event loop work?](#)
7. [How do Promises and `async/await` work?](#)
8. [What are destructuring, spread, and rest?](#)
9. [What are optional chaining \(`?.`\) and nullish coalescing \(`??`\)?](#)
10. [What is the difference between `map`, `filter`, `forEach`, and `reduce`?](#)

11. [What is the difference between shallow copy and deep copy?](#)
 12. [How do arrow functions differ from regular functions?](#)
 13. [How do ES modules work?](#)
 14. [How do you fetch data from an API?](#)
 15. [How do you select DOM elements and attach event listeners?](#)
 16. [What is event bubbling and event delegation?](#)
 17. [How do you handle errors in JavaScript?](#)
 18. [How do you store data in the browser?](#)
 19. [How does prototypal inheritance work? What about classes?](#)
 20. [What modern JavaScript features should juniors know in 2026?](#)
-

1. What are the data types in JavaScript?

Answer: JavaScript has **8 data types** — 7 primitives and 1 non-primitive.

Primitives: `number`, `string`, `boolean`, `undefined`, `null`, `symbol`, `bigint`

Non-primitive: `object` (includes arrays, functions, dates, and plain objects)

```
typeof 42;           // "number"
typeof "hello";      // "string"
typeof true;         // "boolean"
typeof undefined;    // "undefined"
typeof null;         // "object" (historical bug – use
=== null to check)
typeof Symbol();     // "symbol"
typeof 100n;         // "bigint"
typeof {};           // "object"
typeof [];           // "object"
typeof function(){}; // "function" (technically a
callable object)
```

Key concept for juniors: Primitives are stored **by value**; objects are stored **by reference**. Assigning one object variable to another copies the reference, not the object itself.

```
let a = { count: 1 };
let b = a;
b.count = 2;
console.log(a.count); // 2 – both point to the same
object
```

2. What is the difference between `let`, `const`, and `var`?

Answer:

Feature	<code>var</code>	<code>let</code>	<code>const</code>
Scope	Function / global	Block	Block
Hoisting	Yes (<code>undefined</code>)	Yes (TDZ*)	Yes (TDZ*)
Re-declaration	Allowed	Not allowed	Not allowed
Re-assignment	Allowed	Allowed	Not allowed

*TDZ = Temporal Dead Zone — the variable exists but cannot be accessed before its declaration line.

```
// Block scope
if (true) {
  var a = 1;
  let b = 2;
  const c = 3;
}
console.log(a); // 1
console.log(b); // ReferenceError

// const allows mutating object properties
const user = { name: "Alex" };
user.name = "Sam"; // OK
// user = {};      // TypeError

// Classic var loop bug
for (var i = 0; i < 3; i++) {
```

```
setTimeout(() => console.log(i), 0); // 3, 3, 3
}

for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 0); // 0, 1, 2
}
```

Best practice in 2026: Use `const` by default, `let` when reassignment is needed, and avoid `var` in new code.

3. What is the difference between `==` and `===`?

Answer:

- `===` (**strict equality**) compares value **and** type — no type coercion.
- `==` (**loose equality**) coerces types before comparing.

```
5 === "5";    // false
5 == "5";     // true

null === undefined; // false
null == undefined;  // true

0 === false;   // false
0 == false;    // true

[] === [];     // false (different object references)
```

Best practice: Always use `===` and `!==` unless you have a specific reason not to. Type coercion with `==` causes subtle bugs in interviews and production code.

4. What is hoisting?

Answer: During compilation, JavaScript moves **declarations** to the top of their scope. Assignments stay in place.

```
console.log(greet); // [Function: greet] - function
                    // declaration is fully hoisted
greet();            // "Hi"
```

```
function greet() {
  console.log("Hi");
}
```

```
console.log(x); // undefined - var is hoisted,
                // initialized as undefined
var x = 5;
```

```
// console.log(y); // ReferenceError - let is hoisted
// but in TDZ
let y = 10;
```

Summary:

Declaration type	Hoisted?	Usable before line?
<code>function</code> declaration	Yes	Yes (full function)
<code>var</code>	Yes	Yes (as <code>undefined</code>)
<code>let</code> / <code>const</code>	Yes	No (Temporal Dead Zone)
<code>class</code>	Yes	No (TDZ)

5. What is a closure?

Answer: A **closure** is when a function remembers and can access variables from its outer (lexical) scope, even after that outer function has finished executing.

```
function createCounter() {  
  let count = 0;  
  
  return {  
    increment() { return ++count; },  
    getCount() { return count; },  
  };  
}  
  
const counter = createCounter();  
counter.increment(); // 1  
counter.increment(); // 2
```

```
counter.getCount(); // 2
// count is private – not accessible from outside
```

Common use cases:

- Data privacy / encapsulation
- Factory functions
- Event handlers that need context
- Partial application

Interview trap:

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // 3, 3, 3
}
// Fix: use let, or wrap in an IIFE / factory function
```

6. How does the event loop work?

Answer: JavaScript is **single-threaded** — it runs one call stack at a time. The **event loop** coordinates the call stack, Web APIs (browser) / libuv (Node.js), and callback queues so async work does not block the main thread.

Order of execution:

1. Run all synchronous code on the **call stack**
2. When async work completes (timer, fetch, I/O), its callback goes to a **task queue**
3. The event loop pushes queued callbacks onto the stack **only when the stack is empty**
4. **Microtasks** (Promise `.then`, `queueMicrotask`) run before the next macrotask (`setTimeout`, I/O)

```
console.log("1");

setTimeout(() => console.log("2"), 0);

Promise.resolve().then(() => console.log("3"));

console.log("4");

// Output: 1, 4, 3, 2
```

Why it matters: Understanding the event loop explains why `await` yields control, why long synchronous loops freeze the UI, and why Promise callbacks run before `setTimeout(fn, 0)`.

7. How do Promises and `async/await` work?

Answer: A **Promise** represents a value that will be available later — either resolved (success) or rejected (error). States: `pending` → `fulfilled` or `rejected`.

```
function fetchUser(id) {
  return fetch(`/api/users/${id}`)
    .then(res => {
      if (!res.ok) throw new Error(`HTTP
${res.status}`);
      return res.json();
    })
    .then(data => data)
    .catch(err => {
      console.error("Failed:", err.message);
      throw err;
    });
}
```

`async/await` is syntactic sugar over Promises — it makes async code read like synchronous code.

```
async function loadUser(id) {
  try {
    const res = await fetch(`/api/users/${id}`);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return await res.json();
  } catch (err) {
    console.error(err.message);
    return null;
  }
}
```

```
}  
  
}
```

Rules to remember:

- `async` functions **always return a Promise**
 - `await` pauses inside an `async` function until the Promise settles
 - Use `try/catch` around `await` for error handling
 - `Promise.all([...])` runs tasks in parallel; `Promise.allSettled` waits for all regardless of failure
-

8. What are destructuring, spread, and rest?

Answer: Three related ES6+ syntax features used daily in modern JavaScript.

Destructuring — unpack values from arrays or properties from objects:

```
const [first, second, ...remaining] = [10, 20, 30, 40];  
// first = 10, second = 20, remaining = [30, 40]  
  
const { name, age, city = "Unknown" } = { name: "Alex",  
age: 25 };
```

Spread (...) — expand an iterable into individual elements:

```
const nums = [1, 2, 3];
const copy = [...nums];           // shallow copy
const merged = [...nums, 4, 5];

const defaults = { theme: "dark", lang: "en" };
const settings = { ...defaults, lang: "uk" }; //
// override one key
```

Rest (...) — collect remaining items into an array (or object):

```
function sum(...numbers) {
  return numbers.reduce((a, b) => a + b, 0);
}

sum(1, 2, 3); // 6
```

9. What are optional chaining (?.) and nullish coalescing (??)?

Answer: Two operators that make null/undefined handling cleaner — standard in production code since ES2020.

Optional chaining (?.) — safely access nested properties without throwing:

```
const city = user?.address?.city;           // undefined if
any part is null/undefined
const result = obj.method?.();              // undefined if
method doesn't exist
const item = arr?.[0];                      // safe array
access
```

Nullish coalescing (??) — provide a fallback only for `null` or `undefined` (not other falsy values):

```
const count = user.count ?? 0;    // 0 only if count is
null/undefined
const name = input ?? "Guest";

// ?? vs ||
0 || 10;    // 10 - 0 is falsy
0 ?? 10;    // 0 - 0 is not nullish
"" ?? "default"; // ""
"" || "default"; // "default"
```

Combined:

```
const label = config?.settings?.theme ?? "light";
```

10. What is the difference between `map`, `filter`, `forEach`, and `reduce`?

Answer:

Method	Returns	Purpose
<code>forEach</code>	<code>undefined</code>	Run a side effect for each item
<code>map</code>	New array (same length)	Transform each item
<code>filter</code>	New array ($\leq$ length)	Keep items that pass a test
<code>reduce</code>	Single value	Accumulate into one result

```

const products = [
  { name: "Phone", price: 800, inStock: true },
  { name: "Case", price: 20, inStock: true },
  { name: "Charger", price: 35, inStock: false },
];

const inStockNames = products
  .filter(p => p.inStock)
  .map(p => p.name);
// ["Phone", "Case"]

const totalInStock = products
  .filter(p => p.inStock)
  .reduce((sum, p) => sum + p.price, 0);
// 820

```

```
products.forEach(p => console.log(p.name)); // side effect only
```

2026 note: Prefer these over manual `for` loops for readability. Use `for...of` when you need `break`, `continue`, or `await` inside a loop body.

11. What is the difference between shallow copy and deep copy?

Answer:

- **Shallow copy** — a new container, but nested objects still share references.
- **Deep copy** — fully independent clone at every nesting level.

```
const original = { user: { name: "Alex" }, tags: ["js"] };

// Shallow copies
const spread = { ...original };
const sliced = { ...original, tags: [...original.tags] };

spread.user.name = "Sam";
console.log(original.user.name); // "Sam" — nested
```

object is shared

```
// Deep copy (modern, 2026)
const deep = structuredClone(original);
deep.user.name = "Jordan";
console.log(original.user.name); // "Sam" — unchanged
```

When to use what:

Approach	Depth	Good for
<code>{ ...obj }</code>	Shallow	Flat objects, quick copies
<code>[...arr]</code>	Shallow	Flat arrays
<code>structuredClone()</code>	Deep	Nested data, no functions
<code>JSON.parse(JSON.stringify(obj))</code>	Deep (with limits)	JSON-safe data only — loses <code>Date</code> , <code>Map</code> , <code>undefined</code>

12. How do arrow functions differ from regular functions?

Answer:

Feature	Regular function	Arrow function
<code>this</code> binding	Dynamic (call site)	Lexical (from outer scope)
<code>arguments</code> object	Yes	No — use rest <code>...args</code>
Constructor (<code>new</code>)	Yes	No
Hoisting	Yes (declarations)	No

```
const obj = {
  name: "Timer",

  regular() {
    setTimeout(function () {
      console.log(this.name); // undefined — this is
window/global
    }, 100);
  },

  arrow() {
    setTimeout(() => {
```

```
        console.log(this.name); // "Timer" - inherits
this from arrow()
    }, 100);
},
};
```

When to use arrow functions: Callbacks, array methods, short expressions.

When to avoid them: Object methods that need their own `this`, constructors, anything that needs `arguments`.

13. How do ES modules work?

Answer: ES modules (ESM) are the standard way to organize and share code in modern JavaScript. In browsers, use `<script type="module">`. In Node.js, use `"type": "module"` in `package.json` or the `.mjs` extension.

```
// math.js - named exports
export const PI = 3.14159;
export function add(a, b) { return a + b; }

// utils.js - default export
export default function formatDate(date) {
    return date.toISOString().slice(0, 10);
}
```

```
// app.js - imports
import formatDate from "../utils.js";
import { PI, add } from "../math.js";
import * as math from "../math.js";
```

Key differences from CommonJS (`require` / `module.exports`):

ESM	CommonJS
Static imports (analyzed at compile time)	Dynamic <code>require()</code> at runtime
<code>import</code> / <code>export</code>	<code>require()</code> / <code>module.exports</code>
Strict mode by default	Not strict by default
Top-level <code>await</code> supported	No top-level await

2026 reality: New frontend and most modern Node projects use ESM. Know both — legacy Node codebases still use CommonJS.

14. How do you fetch data from an API?

Answer: Use the built-in `fetch` API — available in all modern browsers and Node.js 18+.

```
async function getPosts(limit = 10) {
  const url =
`https://jsonplaceholder.typicode.com/posts?
_limit=${limit}`;

  const response = await fetch(url);

  if (!response.ok) {
    throw new Error(`Request failed: ${response.status}
${response.statusText}`);
  }

  return response.json();
}

// POST with JSON body
async function createPost(title, body) {
  const response = await
fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, body, userId: 1 }),
  });

  if (!response.ok) throw new Error("Create failed");
  return response.json();
}
```

Checklist for production-quality fetch code:

1. Check `response.ok` — fetch does **not** reject on HTTP 4xx/5xx
 2. Set appropriate `headers` (especially `Content-Type` for POST/PUT)
 3. Handle errors with `try/catch`
 4. Consider `AbortController` for timeouts and cancellation
 5. Never expose secrets in frontend code — use a backend proxy for API keys
-

15. How do you select DOM elements and attach event listeners?

Answer: Use `document.querySelector` / `querySelectorAll` for modern selection, then `addEventListener` for events.

```
// Select elements
const form = document.querySelector("#login-form");
const inputs =
document.querySelectorAll("input[name='email']");
const btn = document.getElementById("submit-btn");

// Attach listener
btn.addEventListener("click", (event) => {
  event.preventDefault(); // stop form submit if inside
  a form
  console.log("Clicked!");
});
```

```
// Input event – fires on every keystroke
const search = document.querySelector("#search");
search.addEventListener("input", (e) => {
  console.log("Query:", e.target.value);
});

// Clean up when component is destroyed (SPA frameworks
do this for you)
function onClick() { console.log("once"); }
btn.addEventListener("click", onClick);
btn.removeEventListener("click", onClick);
```

Common selectors:

Method	Returns
<code>getElementById("id")</code>	One element
<code>querySelector(".class")</code>	First match
<code>querySelectorAll("li")</code>	NodeList (all)
<code>closest("form")</code>	Nearest ancestor

2026 note: In React, Vue, or Angular you rarely touch the DOM directly — but interviewers still expect you to understand the underlying browser APIs.

16. What is event bubbling and event delegation?

Answer:

Bubbling — an event fires on the target element, then propagates upward through ancestors to `document`.

Capturing — the opposite direction (top → target). Rarely used directly.

```
// Bubbling phase (default)
child.addEventListener("click", () =>
  console.log("child"));
parent.addEventListener("click", () =>
  console.log("parent"));
// Click child → "child", then "parent"

// Stop propagation
child.addEventListener("click", (e) =>
  e.stopPropagation());
```

Event delegation — attach **one listener** to a parent and use `event.target` to handle events from children. Efficient for dynamic lists.

```
const list = document.querySelector("#todo-list");

list.addEventListener("click", (e) => {
  const btn = e.target.closest("[data-
```

```
action='delete']");  
  if (!btn) return;  
  
  const item = btn.closest("li");  
  item.remove();  
});  
  
// HTML: <li>Buy milk <button data-  
action="delete">x</button></li>  
// Works for items added later – no need to re-attach  
listeners
```

Benefits: Less memory, simpler code for dynamic UIs, one handler for many elements.

17. How do you handle errors in JavaScript?

Answer: Use `try/catch/finally` for synchronous errors and around `await` for async errors. Throw `Error` objects with meaningful messages.

```
function parseConfig(json) {  
  try {  
    const config = JSON.parse(json);  
    if (!config.apiUrl) throw new Error("Missing apiUrl  
in config");  
    return config;  
  } catch (err) {  
    if (err instanceof SyntaxError) {
```



```

        console.error("Invalid JSON:", err.message);
    } else {
        console.error("Config error:", err.message);
    }
    return null;
} finally {
    console.log("Parse attempt finished");
}
}

```

Async error handling:

```

async function loadData() {
    try {
        const data = await fetch("/api/data").then(r =>
r.json());
        return data;
    } catch (err) {
        // Network failure, JSON parse error, or thrown
Error
        showErrorToast(err.message);
        return [];
    }
}

// Unhandled promise rejections – always catch or
return
fetch("/api/data").catch(err => console.error(err));

```

Best practices:

- Throw `Error` instances, not strings: `throw new Error("msg")`
 - Fail fast with clear messages
 - Log errors in development; show user-friendly messages in production
 - Use linters and TypeScript to catch errors before runtime
-

18. How do you store data in the browser?

Answer:

Storage	Capacity	Lifetime	Sent to server?
<code>sessionStorage</code>	~5 MB	Tab session	No
<code>localStorage</code>	~5 MB	Until cleared	No
Cookies	~4 KB	Configurable expiry	Yes (auto)
IndexedDB	Large	Persistent	No

```
// localStorage – persists across browser restarts
localStorage.setItem("theme", "dark");
const theme = localStorage.getItem("theme"); // "dark"
```

```
localStorage.removeItem("theme");

// Store objects — must serialize to JSON
const user = { id: 1, name: "Alex" };
localStorage.setItem("user", JSON.stringify(user));
const saved = JSON.parse(localStorage.getItem("user"));

// sessionStorage — same API, cleared when tab closes
sessionStorage.setItem("step", "2");
```

Important rules:

- Both storages only accept **strings**
- Never store sensitive data (tokens, passwords) in `localStorage` — vulnerable to XSS
- Prefer **httpOnly cookies** for auth tokens (set by the server)
- `localStorage` is synchronous — don't store large payloads

19. How does prototypal inheritance work? What about classes?

Answer: JavaScript uses **prototypal inheritance** — objects can delegate property lookups to another object called their **prototype**.

```
const animal = {  
  speak() { return `${this.name} makes a sound`; },  
};  
  
const dog = Object.create(animal);  
dog.name = "Rex";  
dog.speak(); // "Rex makes a sound" – method found on  
prototype chain
```

Constructor functions + **prototype** (older style):

```
function Person(name) {  
  this.name = name;  
}  
  
Person.prototype.greet = function () {  
  return `Hi, I'm ${this.name}`;  
};  
  
const alex = new Person("Alex");  
alex.greet(); // "Hi, I'm Alex"
```

ES6 **class** syntax (syntactic sugar over prototypes):

```
class Person {  
  #id; // private field (ES2022)  
  
  constructor(name, id) {  
    this.name = name;  
    this.#id = id;  
  }  
}
```

```
greet() {  
  return `Hi, I'm ${this.name}`;  
}  
  
static createGuest() {  
  return new Person("Guest", 0);  
}  
}  
  
class Employee extends Person {  
  constructor(name, id, role) {  
    super(name, id);  
    this.role = role;  
  }  
  
  greet() {  
    return `${super.greet()} - ${this.role}`;  
  }  
}
```

Key interview points:

- `class` does not copy methods onto each instance — they live on the prototype
- `extends` sets up the prototype chain via `super`
- `instanceof` checks the prototype chain: `alex instanceof Person` → `true`

20. What modern JavaScript features should juniors know in 2026?

Answer: Beyond ES6 basics, employers expect familiarity with features shipped in the last few years. You do not need to memorize every proposal — know what they solve and recognize them in code reviews.

Feature	Since	What it does
<code>structuredClone()</code>	2022	Deep clone plain objects, arrays, Dates, Maps, Sets
<code>Array.at(index)</code>	2022	Safe index access — <code>arr.at(-1)</code> gets last item
<code>Object.hasOwn(obj, key)</code>	2022	Safer than <code>obj.hasOwnProperty(key)</code>
Private fields <code>#field</code>	2022	True private class members
Top-level <code>await</code>	ES2022	<code>await</code> at module top level in ESM

Feature	Since	What it does
<code>Object.groupBy()</code> / <code>Map.groupBy()</code>	ES2024	Group array items by key
<code>Promise.withResolvers()</code>	ES2024	Create <code>{ promise, resolve, reject }</code> tuple
<code>Array.toSorted()</code> / <code>toReversed()</code>	ES2023	Non-mutating sort/reverse (return new array)
Iterator helpers <code>(Iterator.from, .map, .filter)</code>	ES2025	Lazy chaining on iterables

```
// Non-mutating array methods – prefer over .sort()
when you need immutability
const nums = [3, 1, 2];
const sorted = nums.toSorted(); // [1, 2, 3]
console.log(nums);               // [3, 1, 2] –
unchanged
```

```
// Group array items
const people = [
  { name: "Alex", dept: "Eng" },
  { name: "Sam",  dept: "Eng" },
  { name: "Jo",   dept: "HR"  },
```

```
];  
const byDept = Object.groupBy(people, p => p.dept);  
// { Eng: [...], HR: [...] }  
  
// Deferred promise — useful in event-driven code  
const { promise, resolve, reject } =  
Promise.withResolvers();  
button.addEventListener("click", () =>  
  resolve("confirmed"));  
const result = await promise;  
  
// Last item without arr[arr.length - 1]  
const last = ["a", "b", "c"].at(-1); // "c"
```

Tooling awareness for 2026 juniors:

- **Package managers:** npm, pnpm, or yarn
- **Bundlers / dev servers:** Vite (most common for new projects)
- **Linting / formatting:** ESLint + Prettier
- **Testing basics:** Vitest or Jest
- **TypeScript:** Increasingly expected — JavaScript with types is the industry default

Quick Study Tips

1. **Practice out loud** — explain closures and the event loop without reading notes.
2. **Write small examples** — one file per topic in the browser console or Node REPL.
3. **Know the "why"** — interviewers care more about trade-offs than memorized definitions.
4. **Read error messages** — `TypeError`, `ReferenceError`, and stack traces tell you exactly what went wrong.
5. **Pair this list with coding exercises** — FizzBuzz, flatten an array, debounce a search input, fetch and render a list.